
Unifor

Flight Routes Check

CSES 1682 — conectividade forte com ordem topológica

Alunos: Enzo Esmeraldo | Gustavo Andrade
Orientador: Prof. Me. Ricardo Carubbi
Disciplina: Resolução de Problemas com Grafos
Centro de Ciências Tecnológicas - CCT
Universidade de Fortaleza - UNIFOR

O problema

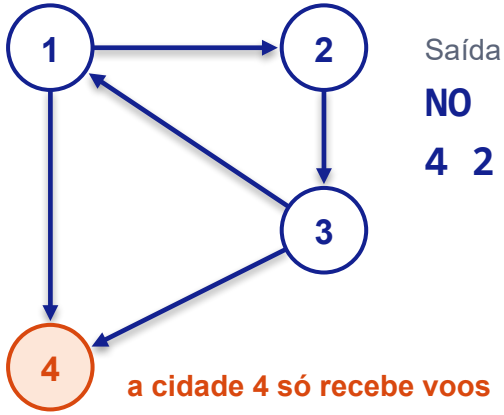
CSES 1682 — Flight Routes Check

n cidades e m voos de mão única (a → b)

De qualquer cidade dá para chegar a qualquer outra?

Sim → YES. Não → NO + um par a b sem rota

$n \le 10^5$ e $m \le 2 \cdot 10^5$ → precisa ser linear



O que realmente se pergunta

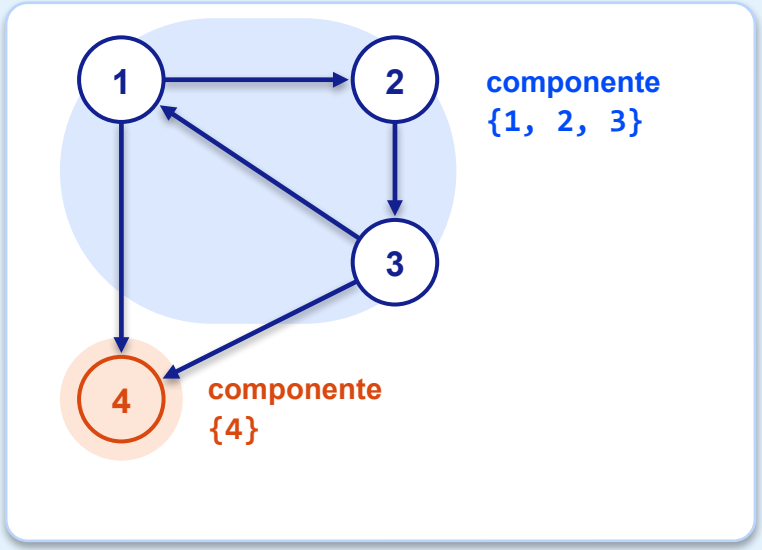
Conectividade forte e componentes f-conexas

Fortemente conexo: para todo par v, w existe caminho v → w e w → v

Componente f-conexa: grupo máximo que se alcança nos dois sentidos

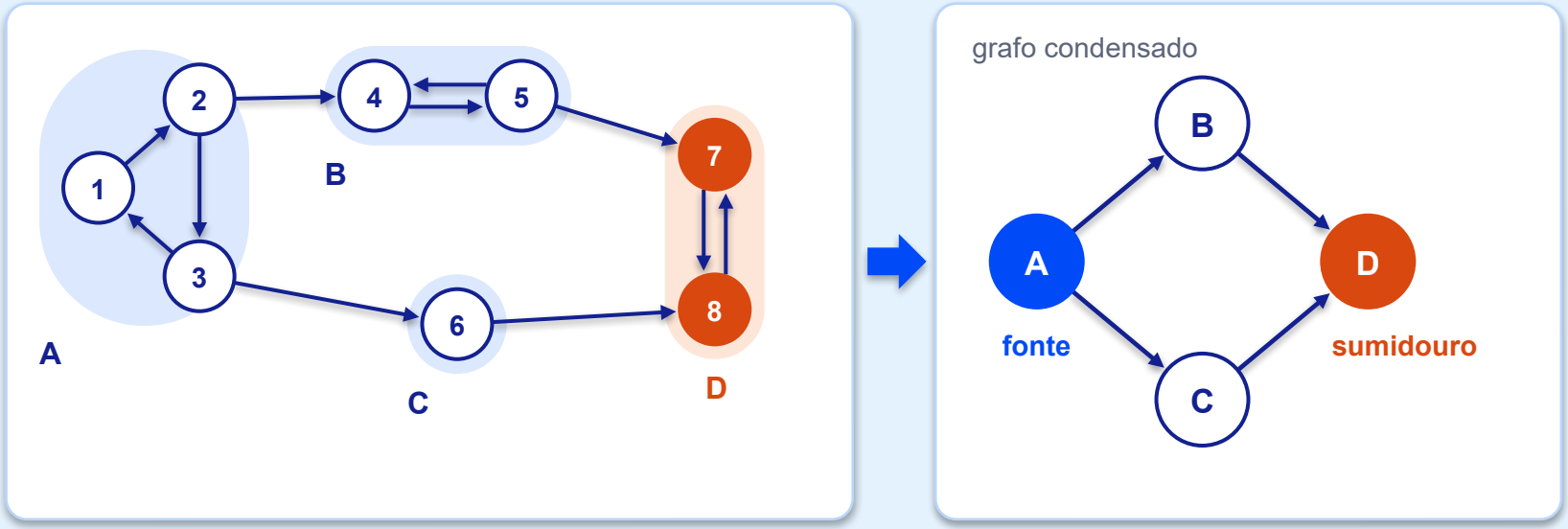
Ignorar a direção dá YES aqui — errado

YES ⇔ exatamente 1 componente f-conexa



Olhando de cima

Cada componente vira um vértice — e sobra um DAG



Entre componentes não há ciclo: **se houvesse, elas seriam uma só.**

Ordem topológica

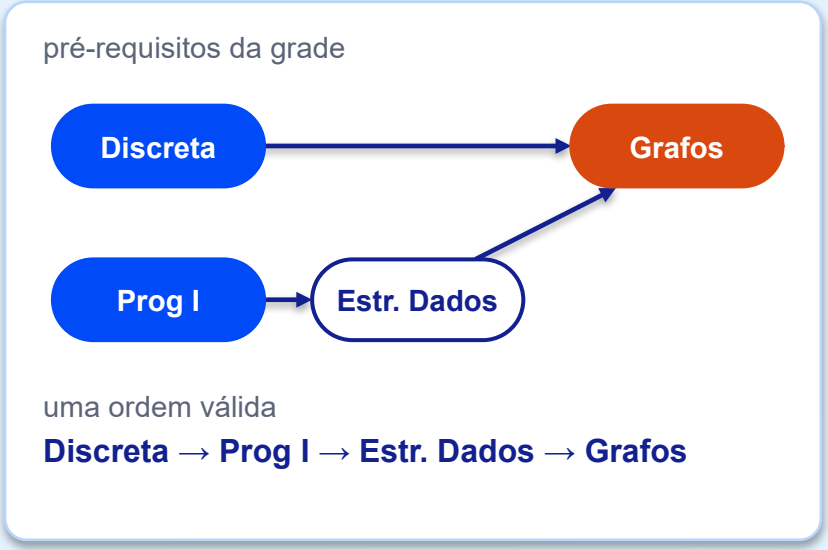
O que é

Enfileirar os vértices de modo que toda seta aponte para a frente

Só existe em DAG — num ciclo, ninguém pode ser o primeiro

Não é única: Prog I antes de Discreta também vale

1º da ordem = fonte; último = sumidouro



Como obter a ordem

Uma DFS e uma pilha: a pós-ordem reversa

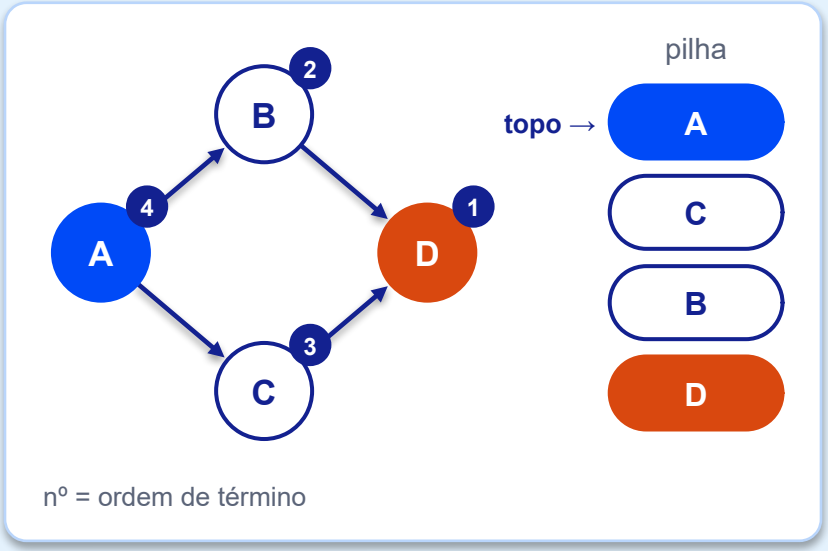
DFS normal; quando um vértice termina, empilha

Ler a pilha do topo = pós-ordem reversa

Em v → w, w termina antes de v

Num DAG: pós-ordem reversa = ordem topológica

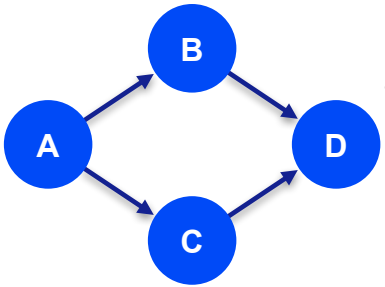
Com ciclos: o 1º vértice cai numa componente fonte



Por que isso basta

Começar a DFS pelo sumidouro

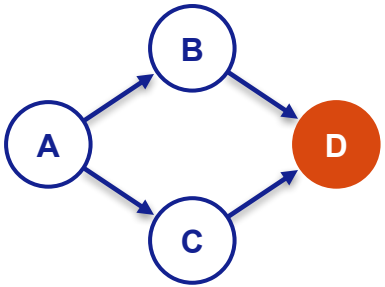
DFS em G começando em A (fonte)



as setas levam a tudo

mistura A, B, C, D ✗

DFS em G começando em D (sumidouro)



não há saída

marca só D ✓

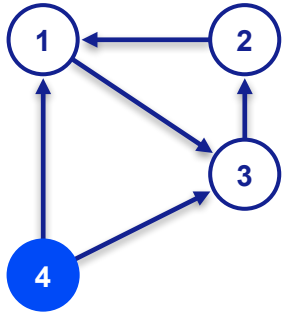
ordem topológica de G^T → **1º = fonte de G^T** → **= sumidouro de G** → **DFS em G não vaza**

Inverter as setas não muda as componentes: um ciclo invertido continua sendo ciclo.

Kosaraju-Sharir

Duas DFS, aplicadas ao exemplo

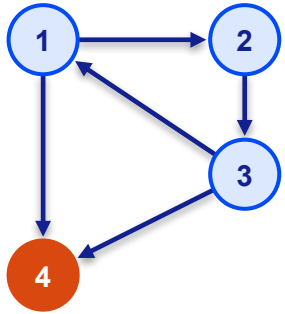
Fase 1 · DFS em G^T



reversePost

4, 1, 3, 2

Fase 2 · DFS em G



nessa ordem

id = [1, 1, 1, 0]

v	ação	id
4	marca {4}	0
1	marca {1,2,3}	1
	(1→4 já marcado)	
3	já marcado	–
2	já marcado	–

count = 2

Cada DFS iniciada na fase 2 **= exatamente uma componente (count++)**

Da contagem à resposta

count e id[ ] já respondem tudo

count == 1 → YES

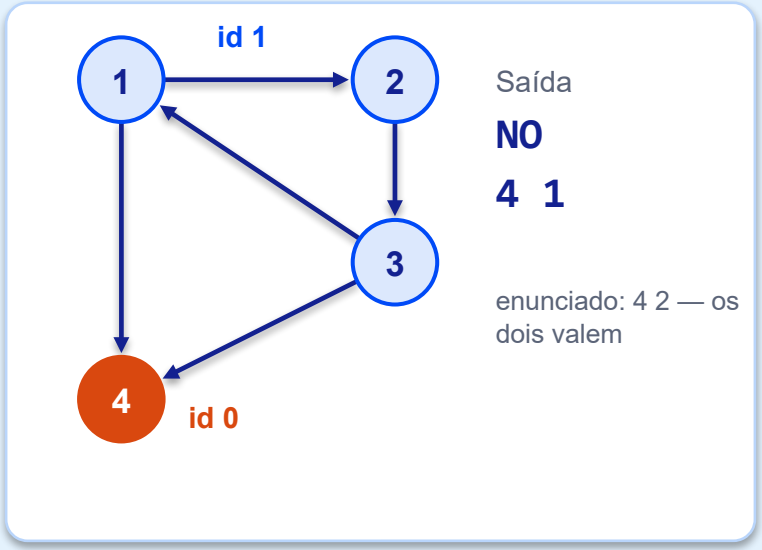
count > 1 → NO

Componente id 0 é um sumidouro: nenhuma seta sai dela

a = qualquer cidade com id 0

b = qualquer cidade com id ≠ 0

Não existe rota de a até b, por construção



Estruturas e custo

O que o código usa (algs4)

Digraph listas de adjacência (Bag) e reverse() → G^T

Stack guarda a pós-ordem reversa

marked[ ], id[ ] visitado e componente

O(V + E) duas DFS + construção de G^T

```
// fase 1: ordem topológica de G^T
DepthFirstOrder ordem =
    new DepthFirstOrder(G.reverse());

// fase 2: DFS em G nessa ordem
for (int v : ordem.reversePost())
    if (!marked[v]) {
        dfs(G, v);   // id[w] = count
        count++;
    }
```

Recursão de até 100.000 níveis

BFS não serve: não tem pós-ordem. DFS mantida, numa Thread com 256 MB de pilha.

Validação e resultados

Testes, complexidade e submissão

8 casos pequenos + 5 de estresse: todos corretos

Cada NO conferido por BFS de a até b

Ciclos ligados num só sentido: NO (sem direção daria YES)

ACCEPTED no CSES — 20/20 testes, pior 0,97 s

caso	esper.	obtido
exemplo oficial	NO	NO 4 1
ciclo único	YES	YES
cidade com laço	YES	YES
caminho 1→2→3	NO	NO 3 1
ciclos, 1 sentido	NO	NO 3 1
cidade isolada	NO	NO 3 1
ciclo 100.000	YES	YES 0,22s
10^5 V, $2 \cdot 10^5$ E	NO	NO 0,27s


Unifor